VOLUNTEERPORTALJAVA

FULL-STACK VOLUNTEER MANAGEMENT SYSTEM

---

1. INTRODUCTION

Volunteer activities play an important role in social development, community service, education, environmental activities, healthcare initiatives, disaster management, and various other areas. Organizations frequently require volunteers for events and social activities, while individuals are interested in finding suitable volunteering opportunities according to their interests and availability.

In many cases, volunteer management is handled using separate tools such as online forms, spreadsheets, emails, messaging applications, and manually maintained records. Although these tools can be useful for small-scale activities, managing a larger number of volunteers and organizations becomes difficult when there is no centralized system.

The VolunteerPortalJava project is developed to address this problem by providing a centralized web-based volunteer management platform. The system connects volunteers with organizations and allows the complete volunteering process to be managed digitally.

The application provides separate functionality for volunteers, organizations, and administrators. Volunteers can discover opportunities and submit applications. Organizations can create and manage opportunities after completing the verification process and can review volunteer applications. Administrators are responsible for managing users, organizations, opportunities, applications, and organization verification.

The system is developed using Java 21 and Spring Boot for the backend, MySQL for database management, Spring Data JPA and Hibernate for persistence, Spring Security and JWT for authentication and authorization, and HTML, CSS, and JavaScript for the frontend. Swagger/OpenAPI is used for API documentation, while Apache PDFBox is used for certificate PDF generation.

The project therefore demonstrates the implementation of a complete full-stack web application with authentication, authorization, database management, REST APIs, business rules, validation, document generation, and frontend-backend integration.

---

2. PROJECT TITLE

VolunteerPortalJava – Full-Stack Volunteer Management System

---

3. PROJECT OBJECTIVE

The primary objective of VolunteerPortalJava is to develop a centralized and secure platform for managing volunteer activities and interactions between volunteers and organizations.

The major objectives of the project are:

1. To provide a centralized platform for volunteer management.
2. To allow volunteers to register and securely log into the system.
3. To allow volunteers to browse available volunteering opportunities.
4. To allow volunteers to submit applications for suitable opportunities.
5. To allow organizations to create and manage volunteering opportunities.
6. To introduce an administrator-based organization verification process.
7. To provide application management and status tracking.
8. To manage volunteer attendance for approved applications.
9. To maintain digital volunteer participation records.
10. To generate certificates for volunteer participation.
11. To generate downloadable PDF certificates.
12. To implement role-based access control.
13. To provide secure REST APIs.
14. To provide Swagger/OpenAPI documentation.
15. To maintain structured relational data using MySQL.
16. To provide a professional and responsive user interface.

---

4. PROBLEM STATEMENT

Traditional volunteer management often involves multiple disconnected methods of communication and record keeping.

Organizations may collect volunteer information through forms, maintain application information in spreadsheets, communicate with volunteers through messaging applications, record attendance manually, and prepare certificates separately.

This creates several difficulties.

First, volunteers may have difficulty discovering available opportunities. There may not be a single platform where they can view all relevant opportunities.

Second, organizations may have difficulty managing a large number of applications. Manually checking applications and maintaining application statuses can become time-consuming.

Third, duplicate or incorrect applications may occur when there is no centralized validation system.

Fourth, organization verification may not be properly controlled. An organization should not automatically receive full privileges without some form of verification.

Fifth, attendance records may be maintained separately from applications, making it difficult to determine whether a volunteer actually participated in an event.

Finally, certificate generation may also be a manual process.

VolunteerPortalJava addresses these issues by integrating the major stages of volunteer management into one application.

---

5. PROPOSED SYSTEM

The proposed system is a full-stack web application that provides a centralized environment for volunteers, organizations, and administrators.

The major workflow of the proposed system is:

Organization Registration

↓

Administrator Verification

↓

Opportunity Creation

↓

Volunteer Discovers Opportunity

↓

Volunteer Application

↓

Organization Reviews Application

↓

Application Approval

↓

Volunteer Participation

↓

Attendance Recording

↓

Certificate Generation

↓

PDF Certificate

The system therefore manages the complete lifecycle of a volunteer opportunity.

---

6. SCOPE OF THE PROJECT

The scope of VolunteerPortalJava includes the following major areas:

6.1 Volunteer Management

The system allows volunteers to register, log in, browse opportunities, submit applications, track application status, view attendance information, and access certificates.

6.2 Organization Management

Organizations can maintain their profiles, undergo administrator verification, create opportunities, manage applications, and manage attendance and certificates.

6.3 Administrator Management

Administrators can manage users, review organizations, verify organizations, and manage major system resources.

6.4 Opportunity Management

Organizations can create opportunities with information such as title, description, category, location, dates, and required volunteer capacity.

6.5 Application Management

Volunteers can apply for opportunities and organizations can review and update application status.

6.6 Attendance Management

Attendance can be recorded for approved applications.

6.7 Certificate Management

Certificate records can be created and PDF certificates can be generated using the configured certificate template.

---

7. USER ROLES

The system contains three major user roles:

1. Volunteer
2. Organization
3. Administrator

Each role has different responsibilities and permissions.

---

8. VOLUNTEER ROLE

The volunteer is the primary participant in volunteering activities.

A volunteer can:

- Create an account
- Log into the system
- Browse opportunities

- View opportunity details
- Apply for opportunities
- Track application status
- View participation information
- View certificates
- Access generated certificates

The volunteer cannot perform administrative operations or manage organizations.

---

9. ORGANIZATION ROLE

Organizations are responsible for providing volunteering opportunities.

An organization can:

- Register an account
- Maintain its organization profile
- Wait for administrator verification
- Create opportunities after verification
- Update opportunities
- Manage opportunities
- View applications
- Approve applications
- Reject applications
- Manage attendance
- Manage certificates

The organization can manage only the resources associated with its account according to the implemented authorization and ownership rules.

---

10. ADMINISTRATOR ROLE

The administrator provides system-level management.

The administrator can:

- Access the administration dashboard
- Manage users
- View organizations
- Verify organizations
- Reject organizations
- Manage opportunities

- Manage applications
- Perform administrative operations

Administrator access is protected through role-based authorization.

Public users cannot simply register themselves as administrators.

---

11. TECHNOLOGY STACK

The project uses the following technologies.

11.1 Java 21

Java 21 is used as the primary programming language for the backend.

Java provides:

- Object-oriented programming
- Strong type safety
- Platform independence
- Large ecosystem
- Extensive enterprise support

Java is suitable for developing maintainable and scalable backend applications.

11.2 Spring Boot

Spring Boot is used to develop the backend application.

It simplifies the development of Spring-based applications by providing automatic configuration, embedded server support, dependency management, and production-oriented features.

Spring Boot is responsible for hosting the REST API and integrating the different backend components.

11.3 Spring Web MVC

Spring Web MVC is used to create REST controllers.

Controllers receive HTTP requests from the frontend and return responses to the client.

For example:

GET requests can retrieve information.

POST requests can create information.

PUT requests can update information.

DELETE requests can remove information.

11.4 Spring Data JPA

Spring Data JPA is used for database persistence.

It reduces the amount of database access code that needs to be written manually.

The application uses repository interfaces for operations such as:

- Finding records
- Saving records
- Updating records
- Deleting records

11.5 Hibernate

Hibernate acts as the JPA implementation used for object-relational mapping.

It maps Java entities to relational database tables.

For example, the Java User entity corresponds to the users table.

This allows the application to work with Java objects while Hibernate manages the conversion between objects and database records.

11.6 MySQL

MySQL is the database used by the project.

It stores information related to:

- Users
- Volunteer profiles
- Organization profiles
- Categories
- Opportunities
- Applications
- Attendance
- Certificates
- Notifications

The project uses a relational database structure to maintain relationships between these entities.

11.7 Spring Security

Spring Security provides authentication and authorization.

It protects backend resources and ensures that users can access only the operations allowed for their roles.

11.8 JWT

JWT stands for JSON Web Token.

JWT is used for maintaining authenticated sessions between the frontend and backend.

After successful authentication, the backend generates a token.

The frontend then sends the token with requests to protected APIs.

11.9 BCrypt

BCrypt is used to hash user passwords.

Passwords are therefore not stored directly as plain text in the database.

11.10 HTML, CSS and JavaScript

The frontend is developed using standard web technologies:

- HTML5
- CSS3
- JavaScript

No frontend framework is required for the current implementation.

JavaScript communicates with the Spring Boot REST API.

11.11 Swagger/OpenAPI

Swagger/OpenAPI is used to document the backend REST APIs.

It allows developers to inspect and test API endpoints through an interactive interface.

11.12 Apache PDFBox

Apache PDFBox is used for PDF certificate generation.

The system uses a certificate template and dynamically adds relevant volunteer and event information.

---

12. SYSTEM ARCHITECTURE

The system follows a layered architecture.

The major layers are:

1. Frontend Layer
2. Controller Layer
3. Service Layer
4. Repository Layer
5. Entity Layer

6. Database Layer

The general architecture is:

Frontend

HTML + CSS + JavaScript

|

| REST API

v

Spring Boot

|

v

Controllers

|

v

Services

|

v

Repositories

|

v

JPA / Hibernate

|

v

MySQL

Security operates alongside the backend:

Spring Security

|

v

JWT Authentication

|

v

Role-Based Authorization

Certificate generation operates through:

Certificate Service

|

v

Apache PDFBox

|

v

PDF Certificate

---

13. FRONTEND ARCHITECTURE

The frontend consists of HTML pages, CSS files, and JavaScript files.

The HTML files provide the page structure.

CSS files control:

- Layout
- Colors
- Typography
- Spacing
- Buttons
- Forms
- Cards
- Responsive behavior

JavaScript files control:

- API communication
- Authentication
- Page logic
- Form submission
- Dynamic content
- Navigation
- Error handling

The frontend communicates with the backend through REST APIs.

---

14. BACKEND ARCHITECTURE

The backend follows a layered architecture.

14.1 Controller Layer

The controller layer handles incoming HTTP requests.

Major controllers include:

- AuthController

- UserController
- VolunteerProfileController
- OrganizationProfileController
- CategoryController
- OpportunityController
- ApplicationController
- AttendanceController
- CertificateController
- NotificationController

14.2 Service Layer

The service layer contains the application's business logic.

Examples include:

- AuthService
- UserService
- VolunteerProfileService
- OrganizationProfileService
- CategoryService
- OpportunityService
- ApplicationService
- AttendanceService
- CertificateService
- CertificatePdfService
- NotificationService

The service layer is important because business rules should not depend only on the frontend.

14.3 Repository Layer

The repository layer communicates with the database through Spring Data JPA.

Repositories are provided for the major entities.

14.4 Entity Layer

Entities represent the application's database objects.

Major entities include:

- User

- VolunteerProfile
- OrganizationProfile
- Category
- Opportunity
- Application
- Attendance
- Certificate
- Notification

14.5 DTO Layer

DTO stands for Data Transfer Object.

DTOs are used to transfer structured data between the frontend and backend.

They also prevent the API from unnecessarily exposing internal entity structures.

---

15. DATABASE DESIGN

The project uses MySQL as its relational database.

The major tables are:

1. users
2. volunteer_profiles
3. organization_profiles
4. categories
5. opportunities
6. applications
7. attendance
8. certificates
9. notifications

---

16. USERS TABLE

The users table stores authentication and basic user information.

It contains information such as:

- User ID
- Name
- Email
- Password

- Role
- Phone and related user information

The role determines whether the user is a volunteer, organization, or administrator.

---

17. VOLUNTEER PROFILES

The volunteer profile is associated with a user account.

It stores volunteer-specific information.

The profile relationship allows authentication information to remain separate from volunteer profile information.

---

18. ORGANIZATION PROFILES

Organization profiles store information about organizations.

Information includes:

- Organization name
- Description
- Address
- Website
- Contact email
- Verification status

Each organization profile is associated with a user account.

---

19. CATEGORIES

Categories provide classification for volunteer opportunities.

They help organizations categorize opportunities and allow opportunities to be organized according to their type.

---

20. OPPORTUNITIES

The opportunity entity represents an event or activity requiring volunteers.

Important information includes:

- Organization
- Category
- Title
- Description
- Location

- Start date
- End date
- Required volunteers
- Status

---

21. APPLICATIONS

Applications connect volunteers with opportunities.

An application contains relationships between:

- Volunteer
- Opportunity

and includes the application status.

Possible statuses include:

- PENDING
- APPROVED
- REJECTED
- CANCELLED
- COMPLETED

---

22. ATTENDANCE

Attendance represents volunteer participation in an opportunity.

Attendance is associated with an application.

The system ensures that an application must be approved before attendance can be recorded.

---

23. CERTIFICATES

Certificates represent proof of volunteer participation.

The system stores certificate-related information such as:

- Volunteer
- Opportunity
- Certificate number
- Issue date

A PDF certificate can then be generated from the stored information.

---

24. NOTIFICATIONS

Notifications provide a structure for communicating important information to users.

They are associated with user accounts and can include notification title, message, type, and read status.

---

25. AUTHENTICATION WORKFLOW

The authentication process follows the following flow:

User

|

| Email + Password

v

Login API

|

v

Spring Security

|

v

Authentication

|

v

JWT Generation

|

v

JWT Returned

|

v

Frontend

|

v

Protected API Request

|

| Authorization: Bearer JWT

v

JWT Filter

|

v

Token Validation

|

v

Authorized Request

This ensures that protected resources are not available to unauthenticated users.

---

26. AUTHORIZATION

Authentication answers:

Who is the user?

Authorization answers:

What is the user allowed to do?

The project implements role-based authorization.

For example:

A volunteer can submit an application.

An organization can manage its opportunities.

An administrator can verify organizations.

The backend remains the final authority for authorization.

---

27. ORGANIZATION VERIFICATION WORKFLOW

Organization verification is an important security and business rule.

The process is:

Organization Registration

|

v

Organization Profile

|

v

PENDING

|

v

Administrator Review

/ \

/ \

v v

VERIFIED REJECTED

Only verified organizations are allowed to perform opportunity management operations according to the implemented business rules.

---

28. OPPORTUNITY WORKFLOW

The opportunity workflow is:

Verified Organization

|

v

Create Opportunity

|

v

Opportunity OPEN

|

v

Volunteers View Opportunity

|

v

Applications

|

v

Event

|

v

Closed / Completed / Cancelled

The backend validates opportunity information and status transitions.

---

29. APPLICATION WORKFLOW

The application process is:

Volunteer

|

v

Select Opportunity

|

v

Submit Application

|

v

PENDING

|

+----------------+

| |

v v

APPROVED REJECTED

|

v

COMPLETED

The application service also prevents duplicate applications.

---

30. ATTENDANCE WORKFLOW

Attendance is created only when the associated application has been approved.

Application

|

v

Approved?

/ \

No Yes

| |

Reject Attendance

|

v

Participation

This ensures that attendance records correspond to legitimate approved participation.

---

31. CERTIFICATE GENERATION WORKFLOW

The certificate process is:

Volunteer Participation

|

v

Certificate Record

|

v

Certificate PDF Service

|

v

Certificate Template

|

v

Dynamic Information Added

|

v

PDF Generated

The PDF generation uses Apache PDFBox.

---

32. CRUD OPERATIONS

CRUD stands for:

Create

Read

Update

Delete

The project uses REST APIs to perform these operations.

For example, opportunity management can follow:

POST /api/opportunities

GET /api/opportunities

GET /api/opportunities/{id}

PUT /api/opportunities/{id}

DELETE /api/opportunities/{id}

However, the project is not limited to simple CRUD.

Business rules are applied before operations are completed.

---

33. VALIDATION

Validation is performed at multiple levels.

Examples include:

- Required fields
- Email format
- Required roles
- Required relationships
- Duplicate records
- Valid dates
- Valid status transitions
- Ownership checks
- Organization verification

This prevents invalid information from being accepted into the system.

---

34. ERROR HANDLING

The application uses a global exception handler.

The purpose is to return consistent API responses when errors occur.

For example:

400 Bad Request

can be returned when invalid input is provided.

This makes error handling easier for the frontend and provides better feedback to users.

---

35. MAJOR DEVELOPMENT CHALLENGES

During development, several technical challenges were encountered.

35.1 Frontend and Backend API Integration

One of the major challenges was ensuring that the frontend and backend used the same API structure.

A backend may return a value using one structure while the frontend expects another structure.

For example, the frontend may expect:

userId

while another representation may contain:

user.id

Such differences can cause the frontend to behave incorrectly even when the database itself contains the correct data.

This demonstrated the importance of maintaining a clear API contract.

35.2 Organization Profile Issue

One important issue occurred with the organization profile.

The organization profile existed in the database and was correctly associated with the user, but the frontend displayed an error indicating that the organization profile could not be found.

This showed that having correct database data is not enough.

The following components must all be synchronized:

Authentication

+

Database

+

Backend Response

+

Frontend JavaScript

The issue required checking how the current logged-in user was identified and how the organization profile response was processed by the frontend.

35.3 Missing JavaScript Helper

Another frontend issue occurred when the volunteer dashboard attempted to use a function called:

getStatusBadgeClass()

but the function was not defined in the page script.

This caused a JavaScript runtime error.

The problem demonstrated that frontend functionality can fail independently of backend functionality.

The required helper functions were added to the dashboard JavaScript.

35.4 Frontend Redesign

The frontend was redesigned to provide a more professional and consistent appearance.

The challenge was to improve:

- typography
- spacing
- layout
- colors
- buttons

- navigation
- responsive behavior

without breaking existing functionality.

Changing a single HTML ID can break JavaScript code if the JavaScript still refers to the old ID.

Therefore, the visual redesign had to be performed while preserving the functional API and JavaScript relationships.

35.5 Security and Authorization

Security required more than simply implementing login.

The system also needed:

- authentication
- role verification
- ownership validation
- organization verification
- API protection
- status validation

For example, an organization should not be able to manage another organization's opportunity.

Similarly, a volunteer should not be able to perform administrative operations.

These checks were therefore implemented in the backend.

---

36. MAIN PROBLEM OF THE CURRENT PROJECT

The main technical challenge of the current project is **maintaining consistency across the complete full-stack system**.

The application contains several connected layers:

MySQL Database

|

v

JPA Entities

|

v

Repositories

|

v

Services

|

v

Controllers

|

v

DTOs / JSON

|

v

JavaScript API Layer

|

v

HTML Interface

A problem in any one layer can affect another layer.

For example:

- Database structure can affect entity mapping.
- Entity mapping can affect service logic.
- Service logic can affect API responses.
- API responses can affect JavaScript.
- JavaScript can affect what the user sees.

Therefore, one of the most important aspects of this project is maintaining consistency between all layers.

---

37. WHY THE PROJECT IS MORE THAN A BASIC CRUD APPLICATION

Although CRUD operations are a fundamental requirement, the system contains additional business functionality.

The project includes:

- Authentication
- JWT
- BCrypt
- Role-based authorization
- Organization verification
- Ownership validation
- Duplicate application prevention
- Application status workflow
- Attendance validation

- Certificate management
- PDF generation
- API documentation
- Validation
- Global exception handling
- Frontend-backend integration
- Responsive frontend

Therefore, CRUD is the foundation, while business rules, security, and workflow management make the application more complete.

---

38. FRONTEND DESIGN APPROACH

The frontend follows a professional editorial and corporate design direction.

The main principles are:

- Minimal design
- Strong typography
- Large headings
- Generous whitespace
- Structured layouts
- Restrained colors
- Subtle hover effects
- Minimal animation
- Responsive design
- Professional forms
- Consistent buttons
- No emojis
- No unnecessary images

The primary visual palette uses:

Black: #111111

Dark: #1A1A1A

Charcoal: #303030

White: #FFFFFF

Off-white: #F7F7F4

Light: #EEEEEA

Accent: #B7C96A

Accent Dark: #8E9F45

Accent Light: #DCE5A7

Muted: #777777

---

39. FRONTEND-BACKEND COMMUNICATION

The frontend never communicates directly with the MySQL database.

Instead, communication takes place through REST APIs.

The process is:

User Action

|

v

HTML / JavaScript

|

v

REST API

|

v

Spring Controller

|

v

Service

|

v

Repository

|

v

MySQL

The result is then returned through the same layers back to the frontend.

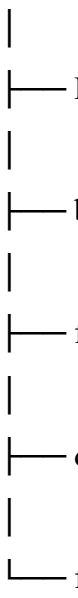
This separation improves maintainability and security.

---

40. PROJECT DIRECTORY STRUCTURE

The major project structure is:

VolunteerPortalJava/



The backend contains Java source code, Spring configuration, resources, entities, controllers, services, repositories, DTOs, and security components.

The frontend contains HTML pages, CSS files, and JavaScript files.

The database folder contains the SQL database schema.

The frontend backup contains a backup copy of the previous frontend.

---

41. TESTING APPROACH

Testing should cover both functional and security requirements.

Important test areas include:

Authentication Testing

- Valid login
- Invalid password
- Invalid email
- Unauthorized API request
- Expired/invalid JWT

Authorization Testing

- Volunteer accessing volunteer features
- Organization accessing organization features
- Admin accessing administrative features
- Unauthorized role attempting restricted operations

CRUD Testing

- Create
- Read
- Update
- Delete

for the major entities.

Application Testing

- Apply to open opportunity
- Prevent duplicate application
- Approve application
- Reject application
- Invalid status transition

Attendance Testing

- Attendance for approved application
- Prevent attendance for unapproved application
- Prevent duplicate attendance

Certificate Testing

- Certificate creation
- Certificate retrieval
- PDF generation

- PDF download/display

---

42. EXPECTED BENEFITS

The system provides several benefits.

For volunteers:

- Easier opportunity discovery
- Centralized applications
- Application tracking
- Digital certificates

For organizations:

- Centralized opportunity management
- Easier application management
- Volunteer tracking
- Attendance management

For administrators:

- Centralized user management
- Organization verification
- Better system control

For the overall system:

- Reduced manual work
- Better data organization
- Improved security
- Better workflow management
- Centralized records

---

43. LIMITATIONS

The current project is primarily designed as an academic full-stack application.

Some advanced production-level features are not part of the current implementation.

Potential areas for future improvement include:

- Email notifications
- Password reset through email
- Real-time notifications
- Advanced analytics
- Volunteer skill matching

- Cloud deployment
- Automated testing pipeline
- Audit logging
- Mobile application
- Advanced reporting
- Production-grade secret management

These should be considered future enhancements rather than existing features unless separately implemented.

---

44. FUTURE ENHANCEMENTS

The project can be extended in several ways.

44.1 Email Notification

The system could send email notifications when:

- An application is approved
- An application is rejected
- An organization is verified
- A certificate is generated

44.2 Advanced Search

Volunteers could search opportunities using:

- Location
- Category
- Date
- Organization
- Availability

44.3 Volunteer Skill Matching

A future recommendation system could match volunteers with opportunities according to their skills and interests.

44.4 Analytics

Organizations could receive statistics about:

- Applications
- Approved volunteers
- Attendance
- Completed events

44.5 Mobile Application

The REST API architecture makes it possible to develop a mobile application in the future.

---

45. HOW TO RUN THE PROJECT

Step 1: Start MySQL

Ensure that MySQL is running.

Step 2: Create the Database

Create:

volunteer_portal

Step 3: Execute Database Schema

Execute:

database/schema.sql

Step 4: Configure Backend

Configure the required database connection and application properties.

Sensitive information such as passwords and JWT secrets should not be committed to public repositories.

Step 5: Open Backend

Navigate to:

backend

Step 6: Build Backend

Run:

./mvnw.cmd clean package

Step 7: Start Spring Boot

Run:

./mvnw.cmd spring-boot:run

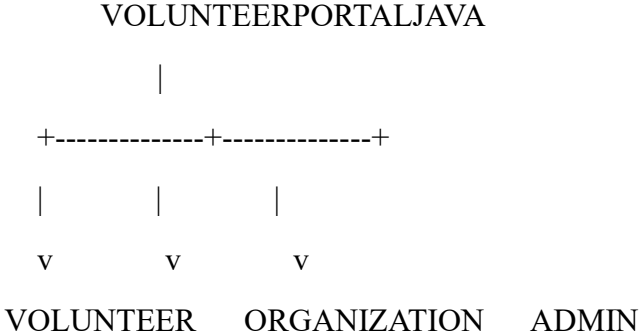
Step 8: Open Frontend

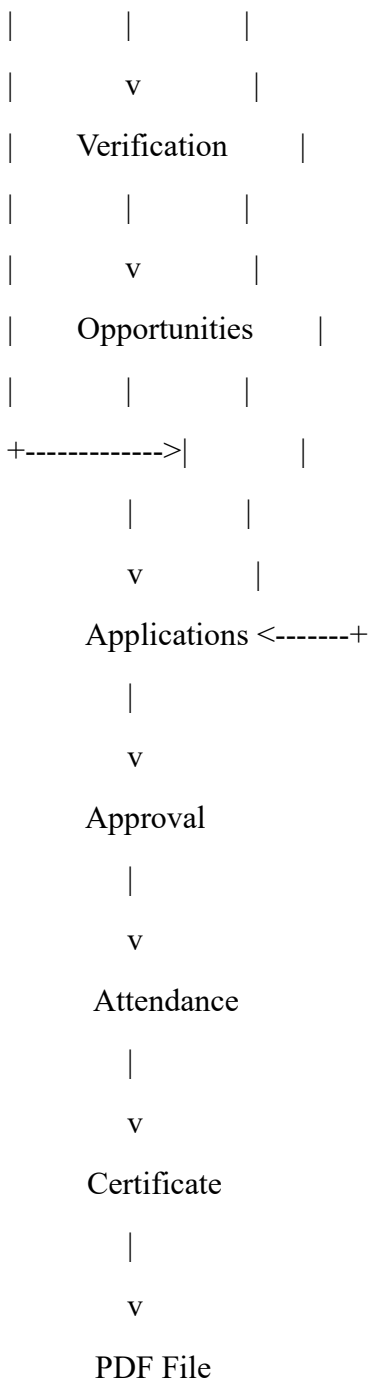
Open the frontend through the configured local development method.

---

46. COMPLETE SYSTEM WORKFLOW

The complete system can be represented as follows:





---

47. CONCLUSION

VolunteerPortalJava is a full-stack web application designed to simplify and centralize volunteer management.

The project combines Java 21, Spring Boot, Spring Data JPA, Hibernate, MySQL, Spring Security, JWT, BCrypt, HTML, CSS, JavaScript, Swagger/OpenAPI, and Apache PDFBox.

The system provides separate functionality for volunteers, organizations, and administrators. Volunteers can discover and apply for opportunities, organizations can manage opportunities and applications, and administrators can control important administrative operations such as organization verification.

The project also implements business rules beyond basic CRUD functionality. These include duplicate application prevention, organization verification, ownership validation, application status transitions, attendance restrictions, and certificate generation.

One of the major technical aspects of the project is the integration between the frontend, REST API, backend services, database, and security layers. During development, several issues were encountered involving frontend-backend data synchronization, missing JavaScript functionality, organization profile retrieval, security rules, and frontend redesign. Resolving these issues provided practical experience in full-stack application development and debugging.

The final system demonstrates the complete development process of a modern web application, starting from database design and backend development to REST API integration, authentication, authorization, frontend development, and PDF document generation.

The project can also serve as a foundation for future improvements such as email notifications, advanced search, analytics, volunteer skill matching, mobile applications, cloud deployment, and automated testing.

Overall, VolunteerPortalJava demonstrates how a centralized full-stack application can be used to organize and manage the complete volunteer participation lifecycle in a structured and secure manner.

---

48. PROJECT TECHNOLOGY SUMMARY

Project Name : VolunteerPortalJava

Programming : Java 21

Backend : Spring Boot 4.1.1

Web Framework : Spring Web MVC

Persistence : Spring Data JPA

ORM : Hibernate

Database : MySQL

Security : Spring Security

Authentication : JWT

Password Security : BCrypt

Validation : Jakarta Validation

API Documentation : Swagger / OpenAPI

PDF Generation : Apache PDFBox 3.0.5

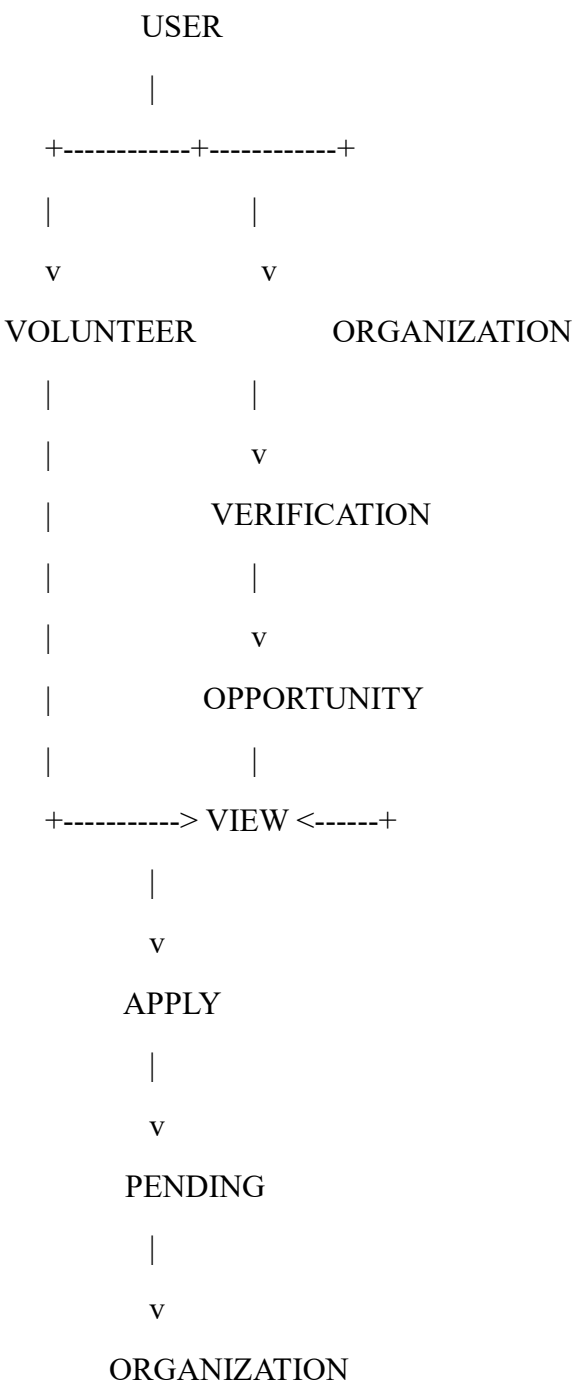
Frontend : HTML5, CSS3, JavaScript

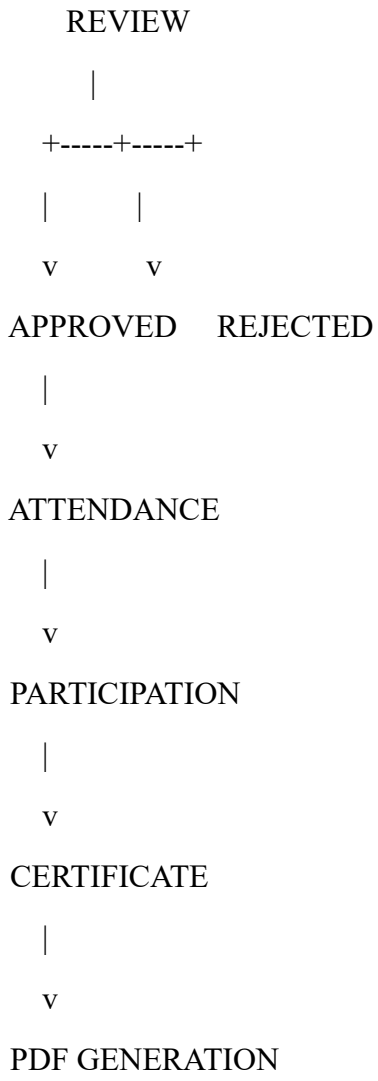
Build Tool : Maven

IDE : Visual Studio Code

---

49. FINAL PROJECT FLOW




REVIEW
APPROVED
REJECTED
ATTENDANCE
PARTICIPATION
CERTIFICATE
PDF GENERATION